

LAPORAN PRAKTIKUM

ABSTRACT CLASS & INTERFACE

Diajukan untuk memenuhi salah satu tugas praktikum Mata kuliah Pemrograman Berorientasi

Objek



Disusun Oleh:

Radithya Mahesa Syabil (251511025)

Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika

Politeknik Negeri Bandung

2026

Identitas

Nama	Radithya Mahesa Syabil
NIM	251511025
Kelas	2A D3

Bagian A - Employee, Manager, dan Sortable

Step 1 - Bangun class Employee

Kode Program Employee.java

```
public class Employee {  
    private String name;  
    private double salary;  
    private int hireDay;  
    private int hireMonth;  
    private int hireYear;  
  
    public Employee(String name, double salary,  
                     int day, int month, int year) {  
        this.name = name;  
        this.salary = salary;  
        this.hireDay = day;  
        this.hireMonth = month;  
        this.hireYear = year;  
    }  
  
    public void print() {  
        System.out.println(name + " " + salary + " " + hireYear());  
    }  
  
    public void raiseSalary(double byPercent) {  
        salary *= 1 + byPercent / 100;  
    }  
  
    public int hireYear() {  
        return hireYear;  
    }  
  
    public double getSalary() {  
        return salary;  
    }  
}
```

Checkpoint 1

Mana state Employee? Mana behavior Employee? Mengapa salary tetap dibuat private walaupun nanti Employee akan mempunyai subclass?

Jawaban:

- **State:**

name, salary, hireDay, hireMonth, hireYear.

- **Behavior:**

print(), raiseSalary(), hireYear(), getSalary().

- **Mengapa salary private:**

Supaya perubahannya hanya lewat method milik Employee, misalnya raiseSalary(). Subclass seperti Manager tidak mengubahnya langsung. Subclass mengakses lewat getSalary() atau super.raiseSalary(). Jadi ini menjaga encapsulation dan mencegah salary diisi nilai yang tidak valid.

Step 2 - Uji beberapa object Employee

Kode Program - EmployeeTest.java

```
public class EmployeeTest {  
    public static void main(String[] args) {  
        Employee[] staff = new Employee[3];  
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);  
        staff[1] = new Employee("Maria Bianchi", 2500000, 1, 12, 1991);  
        staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);  
  
        for (Employee e : staff) {  
            e.raiseSalary(5);  
        }  
        for (Employee e : staff) {  
            e.print();  
        }  
    }  
}
```

Screenshot Output

Output program, sebelum/sesudah kenaikan salary

```
/usr/lib/jvm/java-25-openjdk/bin/java -javaagent:/home,  
+++ Sebelum kenaikan gaji:  
Antonio Rossi 2000000.0 1989  
Maria Bianchi 2500000.0 1991  
Isabel Vidal 3000000.0 1993  
  
+++Setelah kenaikan gaji 5%:  
Antonio Rossi 2100000.0 1989  
Maria Bianchi 2625000.0 1991  
Isabel Vidal 3150000.0 1993  
  
Process finished with exit code 0
```

Gambar 1. Menguji sebelum dan sesudah kenaikan gaji 5%

Step 3 - Buat Manager sebagai subclass Employee

Kode Program - Manager.java

```
import java.util.Calendar;
import java.util.GregorianCalendar;

public class Manager extends Employee {
    private String secretaryName;

    public Manager(String name, double salary,
                    int day, int month, int year) {
        super(name, salary, day, month, year);
        secretaryName = "";
    }

    @Override
    public void raiseSalary(double byPercent) {
        GregorianCalendar today = new GregorianCalendar();
        int currentYear = today.get(Calendar.YEAR);
        double bonus = 0.5 * (currentYear - hireYear());
        super.raiseSalary(byPercent + bonus);
    }

    public String getSecretaryName() {
        return secretaryName;
    }
}
```

Checkpoint 2

Mengapa constructor Manager memanggil super(...)? Pada raiseSalary(), mengapa Manager tetap memanfaatkan super.raiseSalary(...) daripada menulis ulang seluruh proses perubahan salary?

Jawaban:

- **Mengapa super(...):**

field name, salary, dan tanggal bersifat private di Employee. Hanya constructor Employee yang bisa mengisinya. Constructor superclass juga wajib dipanggil, dan harus jadi baris pertama.

- **Mengapa super.raiseSalary(...):**

logika perubahan salary sudah ada di Employee, dan salary private sehingga Manager tidak bisa mengubahnya langsung. Manager cukup menghitung persentase barunya, lalu menyerahkan prosesnya ke superclass. Dengan begitu tidak ada duplikasi kode.

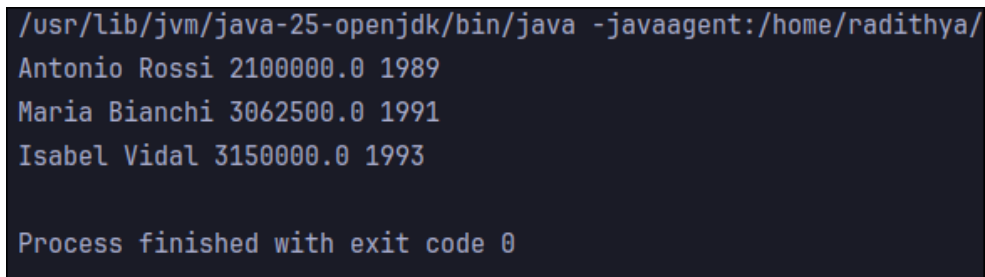
Step 4 - Uji Employee dan Manager dalam satu program

Kode Program - Potongan EmployeeTest.java

```
Employee[] staff = new Employee[3];
staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);
staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

for (Employee e : staff) {
    e.raiseSalary(5);
}
for (Employee e : staff) {
    e.print();
}
```

Screenshot Output



```
/usr/lib/jvm/java-25-openjdk/bin/java -javaagent:/home/radithya/
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993

Process finished with exit code 0
```

Gambar 2. Mengubah salah satu Employee menjadi Manager. Terlihat Manager mendapat kenaikan lebih besar dari 5% pada output.

Checkpoint 3

Lengkapi kalimat dengan bahasa Anda: Manager adalah sebuah _____, sehingga object Manager dapat ditempatkan pada array Employee.

Jawaban:

Manager adalah sebuah Employee, sehingga object Manager dapat ditempatkan pada array Employee.

Step 5 - Buat abstract class Sortable

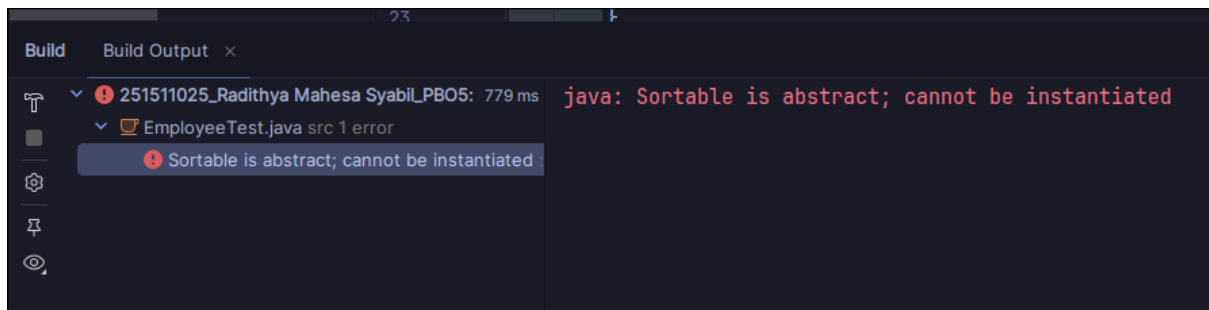
Kode Program - Sortable.java

```
public abstract class Sortable {  
    public abstract int compare(Sortable other);  
}
```

Kode Eksperimen

```
Sortable s = new Sortable();
```

Screenshot Error



Gambar 3. Terjadi error Sortable is abstract; cannot be instantiated

Checkpoint 4

Sortable berguna sebagai class untuk menetapkan kewajiban `compare(...)`, tetapi tidak memiliki bentuk object konkret yang perlu dibuat langsung. Apa bukti compiler yang Anda dapatkan?

Jawaban:

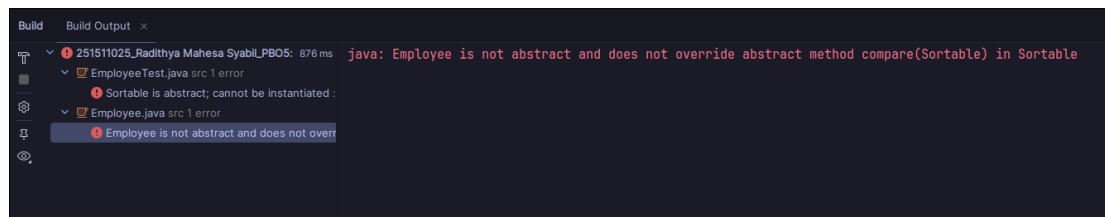
Compiler menolak dengan pesan "Sortable is abstract; cannot be instantiated". Ini membuktikan abstract class tidak bisa dibuat object langsung. Ia hanya dipakai sebagai dasar hierarchy dan menetapkan kewajiban `compare(...)` untuk subclass.

Step 6 - Jadikan Employee subclass dari Sortable

Kode Program - Deklarasi Employee

```
public class Employee extends Sortable {  
    // state, constructor, dan method Employee yang sudah dibuat  
}
```


Screenshot Error



Gambar 4. Terjadi error Employee is not abstract and does not override abstract method compare(Sortable) in Sortable.

Kode Program - Method compare() pada Employee.java

```
@Override
public int compare(Sortable other) {
    Employee otherEmployee = (Employee) other;
    if (getSalary() < otherEmployee.getSalary()) return -1;
    if (getSalary() > otherEmployee.getSalary()) return 1;
    return 0;
}
```

Checkpoint 5

Apa yang diwajibkan Sortable kepada Employee? Mana bagian yang hanya deklarasi pada superclass abstract, dan mana bagian yang menjadi implementasi konkret di Employee?

Jawaban:

- **Yang diwajibkan Sortable:**

Employee harus menyediakan implementasi compare(Sortable other).

- **Hanya deklarasi:**

public abstract int compare(Sortable other); di Sortable. Itu tidak punya body.

- **Implementasi konkret:**

method compare di Employee yang membandingkan salary dan mengembalikan -1, 1, atau 0.

Step 7 - Uji compare(...)

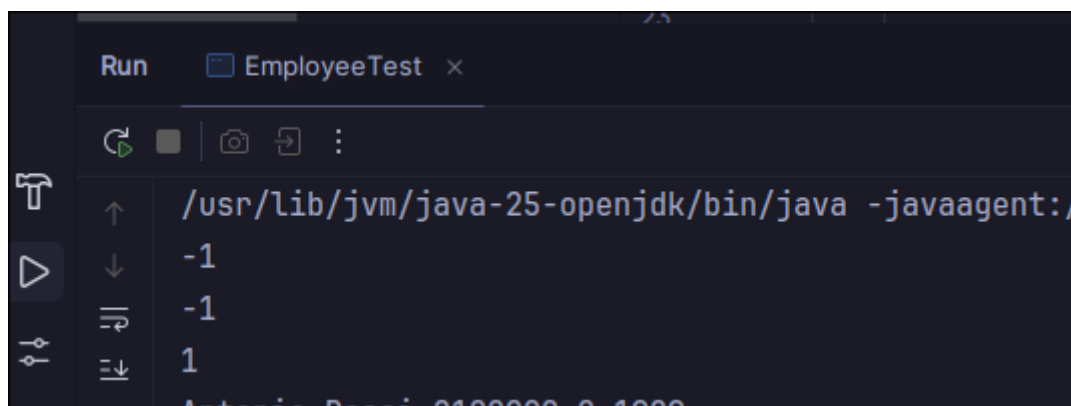
Kode Program - Potongan EmployeeTest.java

```
System.out.println(staff[0].compare(staff[1]));  
System.out.println(staff[1].compare(staff[2]));  
System.out.println(staff[2].compare(staff[0]));
```

Prediksi dan Hasil

Pemanggilan	Prediksi (-1/0/1)	Hasil Run	Arti Hasil (berdasarkan salary)
staff[0].compare(staff[1])	-1	-1	2.000.000 < 2.500.000
staff[1].compare(staff[2])	-1	-1	2.500.000 < 3.000.000
staff[2].compare(staff[0])	1	1	3.000.000 > 2.000.000

Screenshot Output



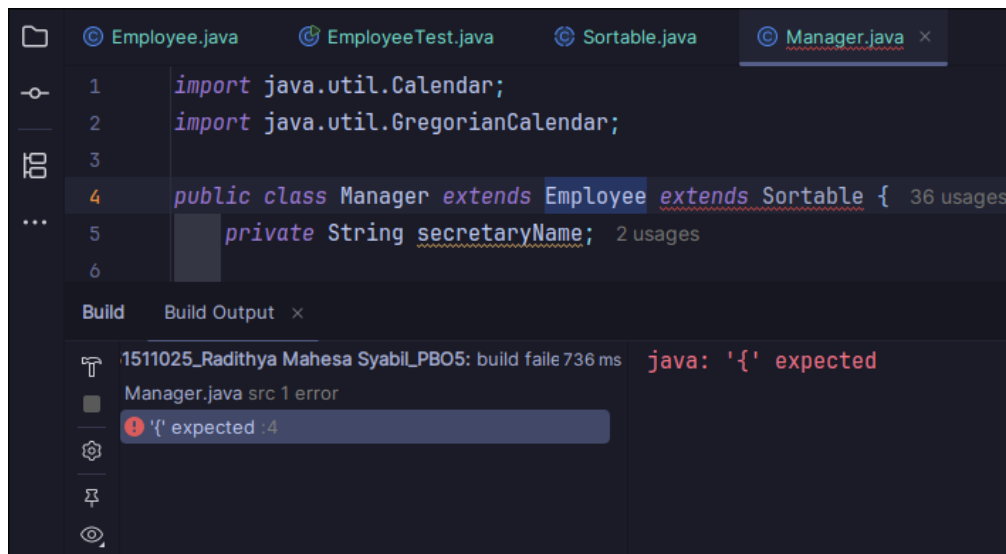
Gambar 5. Hasil compare() pada tiga object Employee: -1, -1, 1. Nilai sesuai urutan salary (kecil, kecil, besar), sehingga compare() berjalan benar.

Step 8 - Uji inheritance chain dan batas extends

Eksperimen A - Kode Program

```
// SALAH - hanya untuk eksperimen  
public class Manager extends Employee extends Sortable {  
}
```

Eksperimen A - Screenshot Error

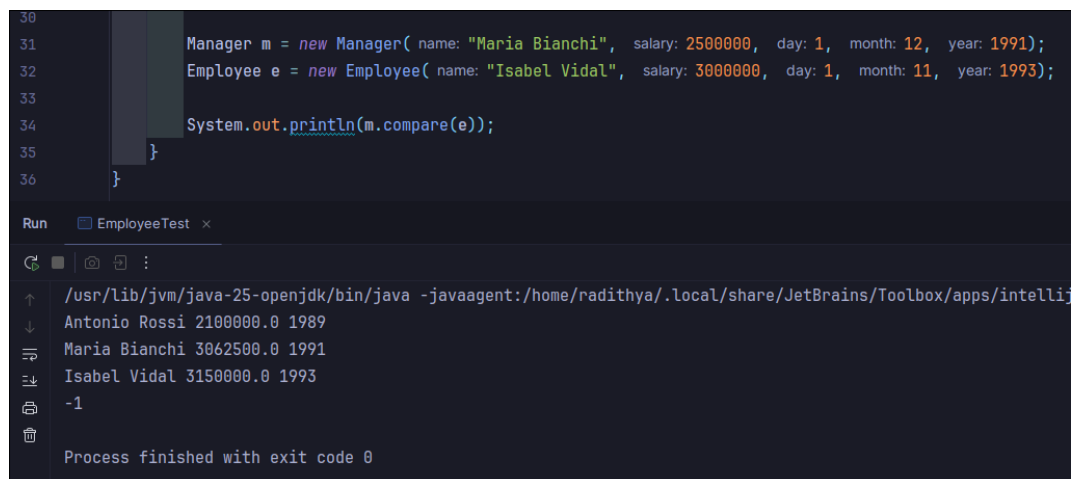


Gambar 6. Terjadi error dengan pesan '{' expected

Eksperimen B - Kode Program

```
Manager m = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);  
Employee e = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);  
System.out.println(m.compare(e));
```

Eksperimen B - Screenshot Output

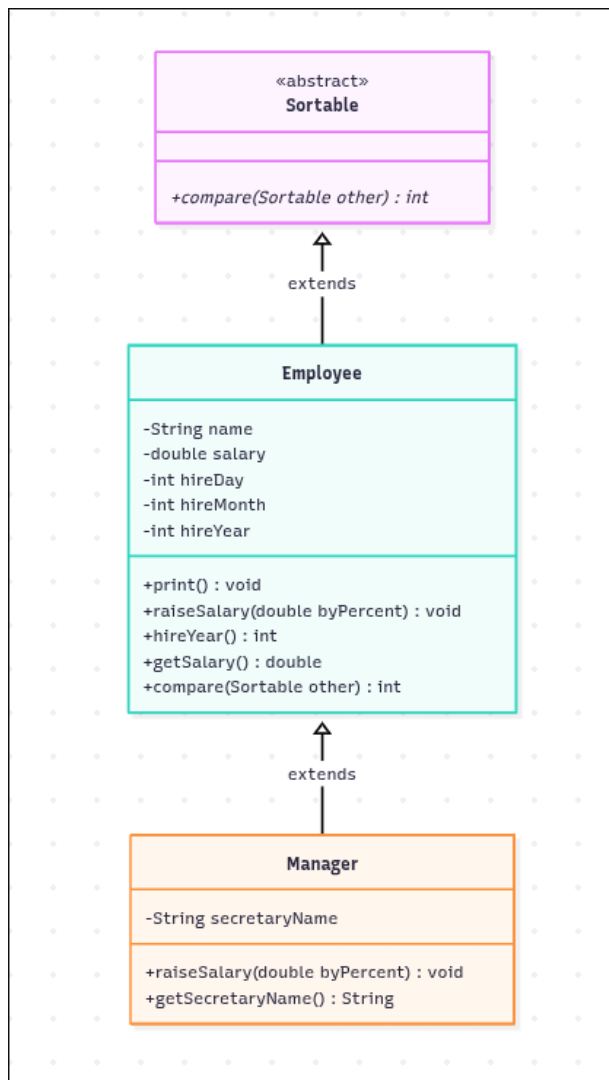


Gambar 7. m.compare(e) pada object Manager menghasilkan -1 walaupun Manager tidak menulis extends Sortable.

Checkpoint 6

Mengapa Manager tetap memiliki compare(...) walaupun tidak pernah menulis extends Sortable?

Gambar rantai class:



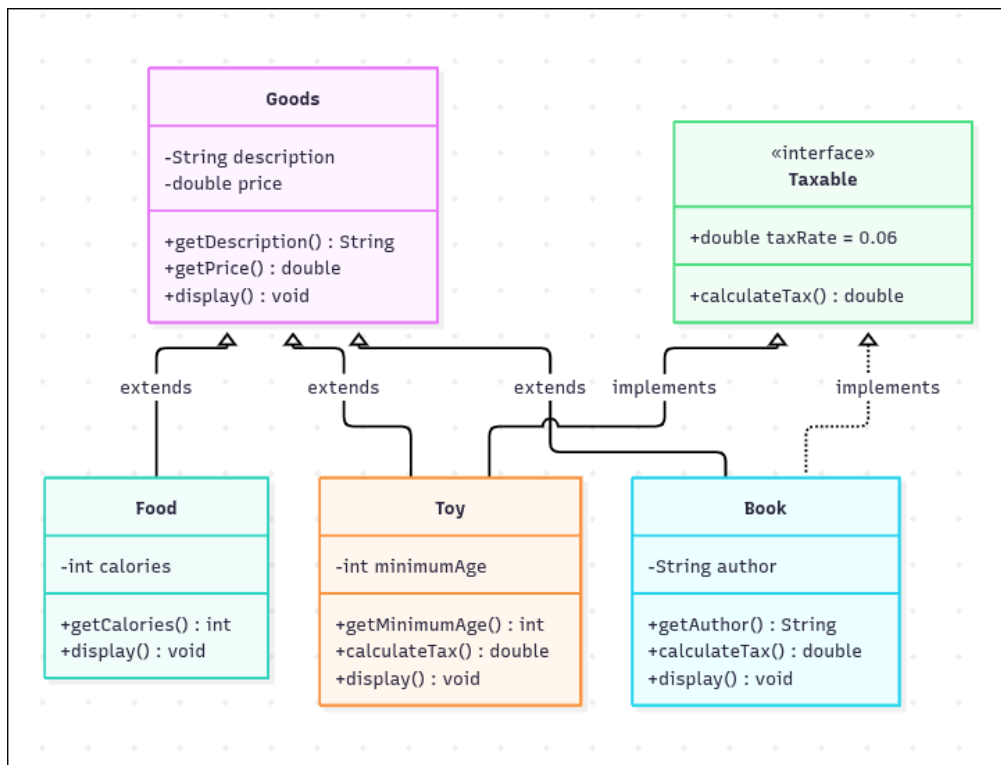
Jawaban:

Manager mewarisi dari Employee, dan Employee mewarisi dari Sortable. Manager otomatis mendapat semua method yang diwarisi Employee, termasuk compare()

Bagian B - Goods dan Taxable

Step 9 - Pahami model sebelum coding

Diagram Hierarchy dan Interface



Gambar 9. Diagram yang menunjukkan hierarki dan interface

Prediksi

State apa yang cukup umum sehingga berada di Goods?

Prediksi:

State description dan price, karena semua barang punya keduanya.

State apa yang hanya dimiliki Food, Toy, dan Book?

Prediksi:

- Food: calories-
- Toy: minimumAge
- Book: author

Mengapa Taxable tidak diterapkan pada semua subclass Goods pada model ini?

Prediksi:

Hanya Toy dan Book yang dikenai pajak. Food tidak dikenai pajak. Taxable itu kemampuan yang hanya dimiliki sebagian class, bukan sifat semua Goods.

Step 10 - Buat class Goods***Kode Program - Goods.java***

```
public class Goods {  
    private String description;  
    private double price;  
  
    public Goods(String description, double price) {  
        this.description = description;  
        this.price = price;  
    }  
  
    public String getDescription() {  
        return description;  
    }  
  
    public double getPrice() {  
        return price;  
    }  
  
    public void display() {  
        System.out.println("Description : " + description);  
        System.out.println("Price : " + price);  
    }  
}
```

Checkpoint 7

Mengapa description dan price diletakkan di Goods, bukan ditulis ulang di Food, Toy, dan Book?

Jawaban:

Description dan price dimiliki semua jenis barang. Dengan menaruhnya di Goods, kodenya ditulis sekali dan diwariskan ke Food, Toy, dan Book. Kalau ditulis ulang di tiap class, kodenya jadi duplikat dan sulit dirawat.

Step 11 - Buat Food sebagai subclass Goods

Kode Program - Food.java

```
public class Food extends Goods {  
    private int calories;  
  
    public Food(String description, double price, int calories) {  
        super(description, price);  
        this.calories = calories;  
    }  
  
    public int getCalories() {  
        return calories;  
    }  
  
    @Override  
    public void display() {  
        super.display();  
        System.out.println("Calories : " + calories);  
    }  
}
```

Screenshot Output



```
public class GoodTest {  
    static void main() {  
        new Food( description: "Baguette", price: 15000, calories: 250).display();  
    }  
}
```

Run GoodTest x

```
/usr/lib/jvm/java-25-openjdk/bin/java -javaagent:/home/radithya/.local/share/  
Description : Baguette  
Price : 15000.0  
Calories : 250  
Process finished with exit code 0
```

Gambar 10. Menunjukkan display() menampilkan state Goods sekaligus state Food

Step 12 - Buat interface Taxable

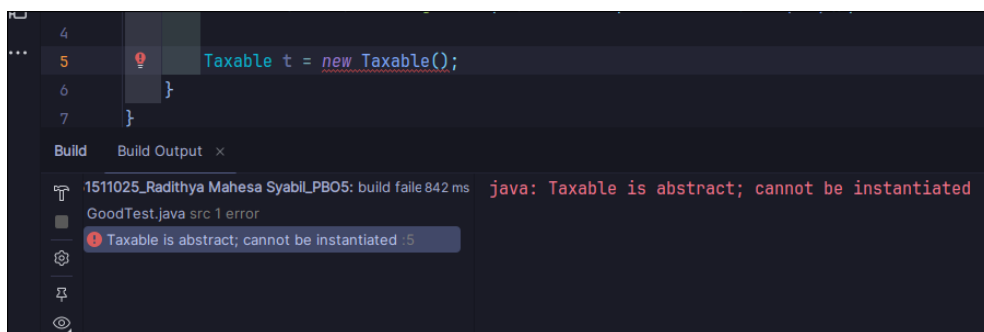
Kode Program - Taxable.java

```
public interface Taxable {  
    double taxRate = 0.06;  
    double calculateTax();  
}
```

Kode Eksperimen

```
Taxable t = new Taxable();
```

Screenshot Error



Gambar 11. Menampilkan output error Taxable is abstract; cannot be instantiated

Checkpoint 8

Apa persamaan hasil eksperimen `new Sortable()` dan `new Taxable()`? Apa perbedaan bentuk deklarasi `Sortable` dan `Taxable`?

Jawaban:

Keduanya tidak bisa diinstansiasi langsung, dan compiler memberi pesan yang sama (is abstract; cannot be instantiated).

Step 13 - Buat Toy yang extends Goods dan implements Taxable

Kode Program - Toy.java

```
public class Toy extends Goods implements Taxable {
    private int minimumAge;

    public Toy(String description, double price, int minimumAge) {
        super(description, price);
        this.minimumAge = minimumAge;
    }

    public int getMinimumAge() {
        return minimumAge;
    }

    @Override
    public double calculateTax() {
        return getPrice() * taxRate;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Minimum age : " + minimumAge);
        System.out.println("Tax : " + calculateTax());
    }
}
```

Checkpoint 9

Pada deklarasi Toy extends Goods implements Taxable, bagian mana yang menyatakan "Toy adalah Goods" dan bagian mana yang menyatakan "Toy memenuhi kontrak Taxable"?

Jawaban:

extends Goods menyatakan "Toy adalah Goods", yaitu mewarisi state dan behavior Goods. Sedangkan implements Taxable: menyatakan "Toy memenuhi kontrak Taxable", yaitu wajib menyediakan calculateTax().

Step 14 - Buat Book yang extends Goods dan implements Taxable

Kode Program - Book.java

```
public class Book extends Goods implements Taxable {
    private String author;

    public Book(String description, double price, String author) {
        super(description, price);
        this.author = author;
    }

    public String getAuthor() {
        return author;
    }

    @Override
    public double calculateTax() {
        return getPrice() * taxRate;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Author : " + author);
        System.out.println("Tax : " + calculateTax());
    }
}
```

Perbandingan Toy dan Book

Bandingkan Toy dan Book: keduanya punya parent yang sama dan contract yang sama, tetapi state spesifiknya berbeda.

Perbandingan:

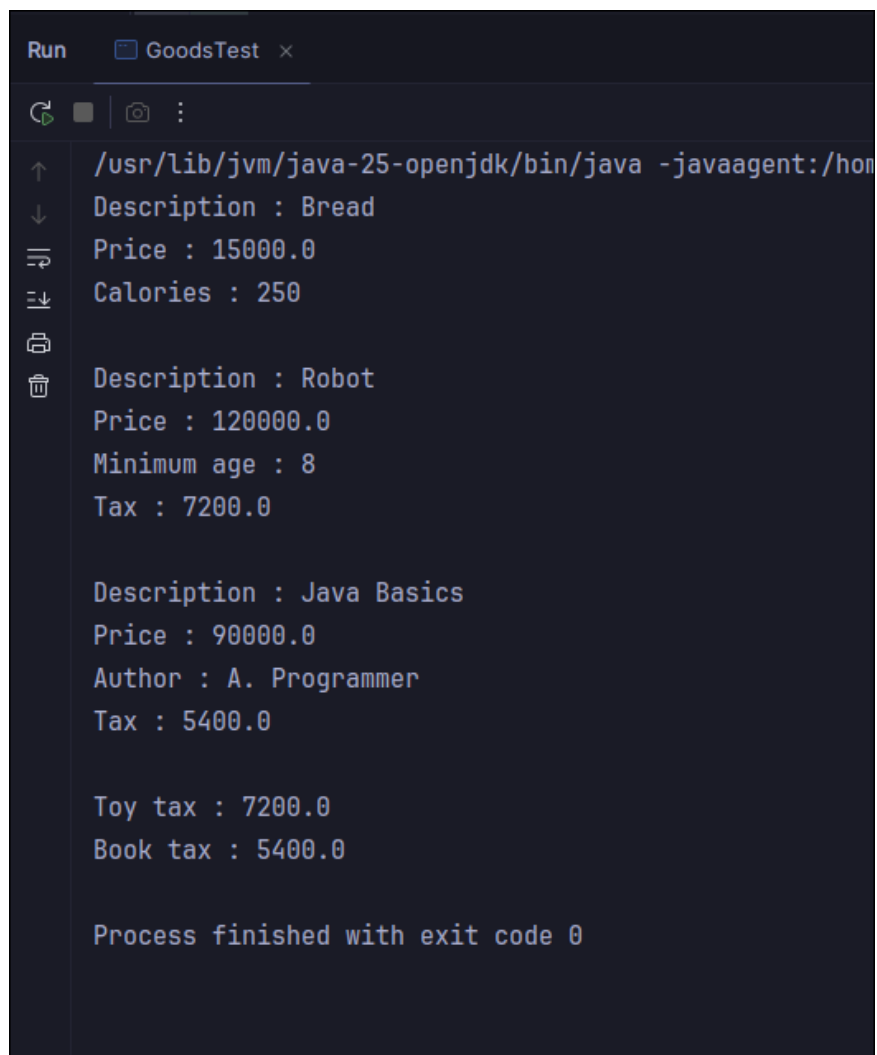
Toy dan Book sama-sama extends Goods implements Taxable. Perbedaannya hanya state spesifik seperti minimumAge dan author.

Step 15 - Uji hierarchy dan contract secara utuh

Kode Program - GoodsTest.java

```
public class GoodsTest {  
    public static void main(String[] args) {  
        Food food = new Food("Bread", 15000, 250);  
        Toy toy = new Toy("Robot", 120000, 8);  
        Book book = new Book("Java Basics", 90000, "A. Programmer");  
  
        food.display();  
        System.out.println();  
        toy.display();  
        System.out.println();  
        book.display();  
        System.out.println();  
  
        Taxable taxableToy = toy;  
        Taxable taxableBook = book;  
        System.out.println("Toy tax : " + taxableToy.calculateTax());  
        System.out.println("Book tax : " + taxableBook.calculateTax());  
    }  
}
```

Screenshot Output

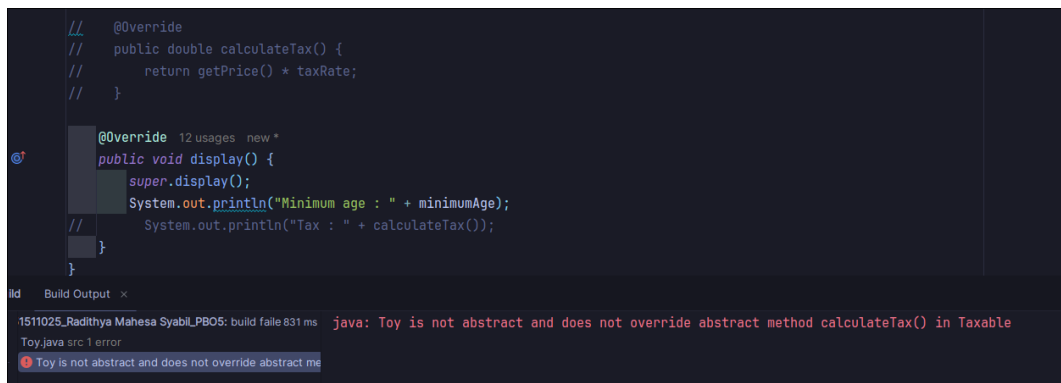


```
Run GoodsTest x
/usr/lib/jvm/java-25-openjdk/bin/java -javaagent:/hom
Description : Bread
Price : 15000.0
Calories : 250
Description : Robot
Price : 120000.0
Minimum age : 8
Tax : 7200.0
Description : Java Basics
Price : 90000.0
Author : A. Programmer
Tax : 5400.0
Toy tax : 7200.0
Book tax : 5400.0
Process finished with exit code 0
```

Gambar 12. Hasil outuput untuk Step 15

Step 16 - Sengaja langgar kontrak interface

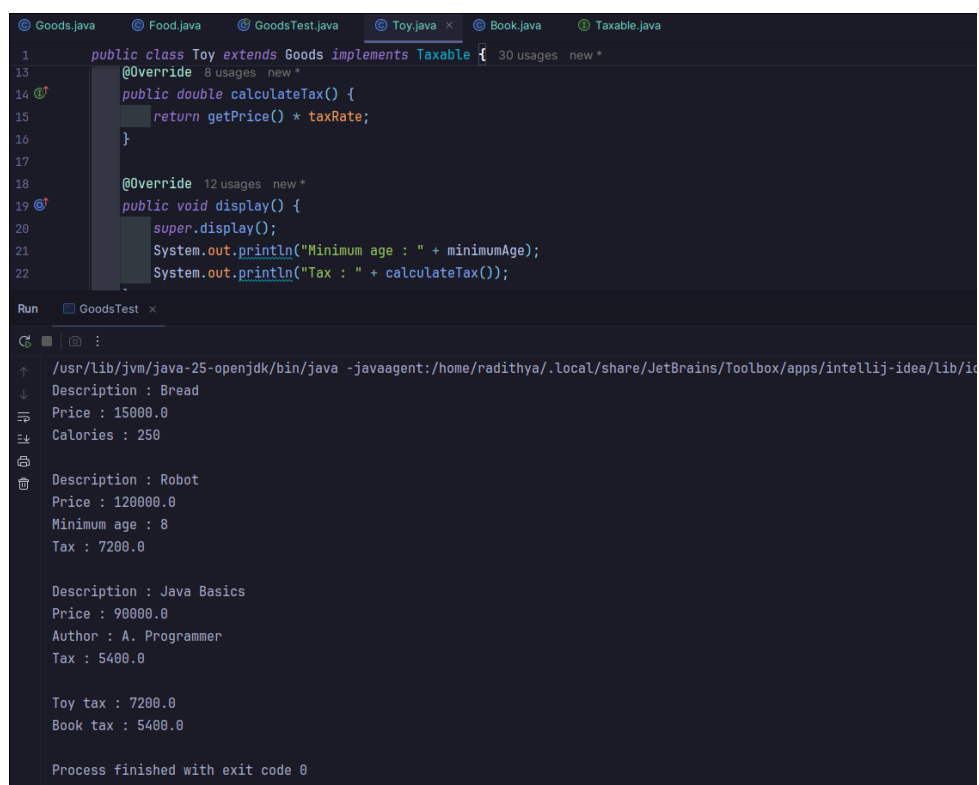
Screenshot Error



Gambar 13. Menghasilkan output error Toy is not abstract and does not override abstract method calculateTax() in Taxable

Screenshot Project Kembali Compile

Kembalikan calculateTax() ke kondisi benar dan pastikan seluruh project kembali compile.



Gambar 14. calculateTax() sudah dikembalikan ke Toy, dan GoodsTest berjalan tanpa error

Checkpoint 10

Keyword implements bukan sekadar label. Ketika sebuah class implements interface, compiler menuntut class konkret tersebut memenuhi method yang dideklarasikan interface.

Bukti dan penjelasan:

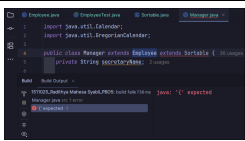
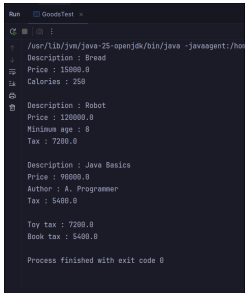
Compiler meminta Toy sebagai class konkret, mengimplementasikan calculateTax() karena ia menyatakan implements Taxable. implements bukan hanya sekadar label. Ia merupakan janji yang diperiksa compiler.

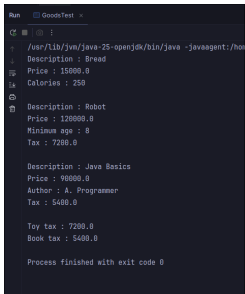
Step 17 - Bandingkan abstract class dan interface dari pengalaman code

Pertanyaan	Sortable (abstract class)	Taxable (interface)
Dapat dibuat object langsung?	Tidak. new Sortable() ditolak: Sortable is abstract; cannot be instantiated.	Tidak. new Taxable() ditolak dengan pesan yg sama.
Menggunakan extends atau implements?	extends (Employee extends Sortable).	implements (Toy extends Goods implements Taxable).
Dapat mempunyai implementasi method konkret?	Bisa. Abstract class boleh punya method abstract dan method biasa yang berbody.	Pada kode praktikum ini, calculateTax() hanya deklarasi tanpa body.
Peran utamanya pada kasus praktikum?	Dasar hierarchy yang mewajibkan compare(). Manager ikut mendapatkannya lewat rantai Sortable <- Employee <- Manager.	Kontrak kemampuan pajak, yang hanya dimiliki Toy dan Book, bukan Food.
Apakah class dapat mempunyai parent class lain sekaligus?	Tidak karena class Java hanya boleh extends satu class. Employee sudah memakai satu slot parent itu untuk Sortable.	Ya. Toy sudah punya parent Goods dan tetap bisa implements Taxable nya.

Tabel Skenario Uji T5-01 s.d. T5-08

Step 18 - Uji sebelum menyatakan selesai

ID	Skenario	Expected Result	Status (PASS/FAIL)	Bukti
T5-01	new Sortable() dicoba	Compiler menolak instansiasi abstract class	PASS	
T5-02	Employee extends Sortable tanpa compare()	Compiler menuntut implementasi abstract method	PASS	
T5-03	Employee mengimplementasikan compare()	Hasil perbandingan -1/0/1 sesuai salary	PASS	
T5-04	Manager hanya extends Employee	Manager tetap memiliki compare() melalui inheritance chain	PASS	m.compare(e) menghasilkan -1
T5-05	Manager extends Employee extends Sortable	Compiler menolak multiple extends class	PASS	
T5-06	Toy/Book implements Taxable	calculateTax() dapat dipanggil	PASS	
T5-07	calculateTax() dihapus dari Toy	Compiler menolak class konkret yang tidak memenuhi contract	PASS	

ID	Skenario	Expected Result	Status (PASS/FAIL)	Bukti
T5-08	Food diuji sebagai Goods	State/behavior Goods tersedia melalui inheritance	PASS	

Self Assessment

Pernyataan	Skor 1-5
Saya memahami mengapa abstract class tidak dapat diinstansiasi.	4
Saya dapat membuat abstract method dan implementasinya di subclass.	4
Saya memahami inheritance chain tanpa menulis extends lebih dari satu class.	4
Saya dapat membuat interface dan class yang implements interface tersebut.	3
Saya memahami bahwa implements berarti memenuhi contract method.	3
Saya dapat membedakan kapan konsep lebih cocok menjadi abstract class dan kapan menjadi interface.	3

Jawaban Refleksi

Refleksi 1

Mengapa Sortable berguna walaupun object Sortable tidak pernah dibuat?

Jawaban:

Karena ia menetapkan kewajiban compare(...) untuk semua subclass. Kode lain bisa memperlakukan objek sebagai Sortable tanpa tahu jenis konkretnya.

Refleksi 2

Apa yang terjadi ketika Employee extends Sortable tetapi belum membuat compare()?

Jawaban:

Compile gagal dengan pesan Employee is not abstract and does not override abstract method compare(Sortable) in Sortable. Employee adalah class konkret, jadi semua abstract method harus diimplementasikan.

Refleksi 3

Mengapa Manager tetap dapat menggunakan compare() tanpa menulis extends Sortable?

Jawaban:

Manager extends Employee, dan Employee extends Sortable. Method diwariskan sepanjang rantai: Sortable <- Employee <- Manager. Buktinya m.compare(e) berjalan dan menghasilkan -1.

Refleksi 4

Mengapa class Manager extends Employee extends Sortable tidak diperlukan dan tidak valid di Java?

Jawaban:

Karena Java hanya mengizinkan satu superclass per class, dan compiler menolak dengan '{' expected tadi. Sortable juga sudah terwarisi lewat Employee.

Refleksi 5

Apa yang dijanjikan Toy ketika menulis implements Taxable?

Jawaban:

Toy berjanji menyediakan implementasi calculateTax().

Refleksi 6

Mengapa Goods lebih tepat menjadi superclass bagi Food/Toy/Book, sedangkan Taxable lebih tepat menjadi interface pada model ini?

Jawaban:

- Food, Toy, dan Book memang jenis Goods (relasi is-a). Mereka berbagi state description dan price serta method display().
- Taxable adalah kemampuan yang hanya dimiliki sebagian class, yaitu Toy dan Book.
- Toy dan Book sudah memakai satu slot parent untuk Goods, dan interface bisa dipasang tanpa mengganggu itu.

Refleksi 7

Apa bukti dari compiler bahwa interface benar-benar merupakan contract?

Jawaban:

Saat `calculateTax()` dihapus dari `Toy` yang masih implements `Taxable`, compiler menolak dengan pesan `does not override abstract method calculateTax() in Taxable`.

Refleksi 8

Lengkapi dengan bahasa Anda sendiri: "Abstract class membantu saya ketika ..., sedangkan interface membantu saya ketika ..."

Jawaban:

Abstract class membantu saya ketika beberapa class punya dasar yang sama seperti state dan behavior dan perlu memaksa subclass mengimplementasikan method tertentu, sedangkan interface membantu saya ketika class dari hierarchy berbeda perlu memenuhi kontrak behavior yang sama tanpa harus punya parent yang sama.